MRIGMAREECHA

AI-POWERED COLLECTIVE CYBER DEFENSE

mitigata

FULL STACK CYBER RESILIENCE

One signal can change *the way we defend.*

Turning controlled-deception signals into explainable threat assessments, reviewable defensive rules, and traceable incident evidence.

THE PROJECT

MRIGMAREECHA

Detect carefully. Understand clearly. Defend together.

— THE IDEA —

We followed the signal. *It led to a bigger idea.*

From isolated alerts to shared defensive intelligence.

Security teams investigate fragmented alerts, repeat manual work, and struggle to turn one incident into a reusable defensive lesson.

Mrigmareecha connects controlled deception, AI-assisted analysis, human-reviewed defensive rules, and tamper-evident records in one proposed workflow.

One finding. A clearer response. A reusable lesson.

TEAM NAME

360degree

TEAM MEMBERS

Vipul Mhatre

Samarth Pachpile

CHALLENGE

AI in Cybersecurity

PROJECT REPOSITORY

GitHub Link: Mrigmareecha

Executive Summary

The name **Mrigmareecha** draws on Mareecha's golden-deer form in the Ramayana, which served as a deceptive distraction. In this project, the metaphor represents a defensive use of deception: controlled decoys create observable signals that help defenders detect and investigate unexpected interactions.

Security teams often face fragmented alerts, repetitive triage, limited cross-organization threat sharing, and incident evidence spread across different tools. Mrigmareecha proposes a workflow that connects four capabilities:

1. **Controlled deception:** deploy safe decoy services and canary artifacts in authorized environments.
2. **AI-assisted triage:** enrich events, summarize evidence, estimate severity, and recommend defensive next steps.
3. **Governed collective intelligence:** turn validated observations into sanitized defensive rules that participants can review and adopt.
4. **Tamper-evident evidence:** preserve a traceable record of events, analyst decisions, rule versions, and actions.

The hackathon MVP will demonstrate a complete workflow in a controlled lab: an event is generated, ingested, analyzed, displayed, converted into a candidate rule, reviewed by a human, and recorded in an exportable evidence bundle.

Submission thesis
Mrigmareecha does not replace a security operations team. It aims to reduce the distance between a suspicious signal and a reviewable, reusable defensive action.

Proposal at a Glance

Challenge	AI in Cybersecurity
Core users	Security analysts, IT administrators, and security teams at small and mid-sized organizations
Primary input	Decoy-service events, canary-token activations, and authorized security telemetry
AI contribution	Contextual enrichment, prioritization, evidence summarization, and defensive-rule suggestions
Human control	Analysts approve rule promotion and consequential response actions
MVP output	Alert, explanation, candidate rule, audit trail, and evidence export
Future extension	Integrations with SOC platforms, approved Mitigata workflows, and cyber-risk review systems

Contents

Executive Summary **2**

1 Project Concept **5**

1.1 Why Mrigmareecha? 5

1.2 One-Line Description 5

1.3 Problem Statement 5
1.4 Project Goals 5

2 Proposed Solution 5
2.1 Controlled Deception 6
2.2 AI-Assisted Investigation 6
2.3 Collective Defensive Learning 6
2.4 Evidence and Risk Review 6
2.5 Core Differentiators 7

3 Target Users and Use Cases 7
3.1 Primary Users 7
3.2 Illustrative Scenarios 7
3.2.1 Unexpected Access to a Decoy Service 7
3.2.2 Canary Artifact Activation 7
3.2.3 Reusable Defensive Pattern 7
3.2.4 Evidence Export 7

4 System Architecture 8
4.1 High-Level Architecture 8
4.2 MVP Component Design 9
4.3 Gordon AI Integration Boundary 9

5 End-to-End Event Lifecycle 9
5.1 Step-by-Step Workflow 9
5.2 Illustrative Event Record 10
5.3 Structured Analysis Output 11

6 AI-Assisted Threat Analysis 11
6.1 Role of AI 11
6.1.1 Deterministic Processing 11
6.1.2 Model-Assisted Interpretation 12
6.1.3 Prompt and Data Security 12
6.2 Threat Assessment Dimensions 12

7 Collective Defense and Rule Sharing 12
7.1 From One Finding to Reusable Intelligence 12
7.2 Proposed Rule Lifecycle 13
7.3 Example Rule Representation 14
7.4 Why Not Automatically Push Every Rule? 15

8 Evidence Integrity and Cyber-Risk Workflows 15

8.1 Evidence as a First-Class Output 15
8.2 Tamper-Evident Audit Trail 15
8.3 Parametric Insurance: Future Extension 16

9 Hackathon MVP Scope **17**
9.1 What We Will Build 17
9.2 Proposed Technology Stack 18

10 Implementation Plan **18**
10.1 Suggested Build Sequence 18

11 Security, Privacy, and Governance **18**
11.1 Threat Model 19
11.2 Operational Guardrails 19
11.3 Privacy-Aware Sharing 19

12 Expected Impact and Evaluation **19**
12.1 Evaluation Metrics 20
12.2 Proposed Success Criteria 20
12.3 Potential Real-World Value 20

13 Alignment with CyberForge **20**

14 Conclusion **21**

References and Implementation Notes **21**

1 Project Concept

1.1 Why Mrigmareecha?

The name is inspired by the golden deer episode in the Ramayana. The metaphor is adapted for defensive deception: deliberately placed decoys can make unexpected interactions visible to the defenders who own and monitor them.

The project is not designed to deceive legitimate users, collect unnecessary personal information, or retaliate against suspected attackers. It uses controlled, authorized deception assets to generate useful defensive signals.

1.2 One-Line Description

> Mrigmareecha is an AI-assisted collective cyber defense platform that transforms controlled-deception signals into explainable security findings, reviewed defensive guidance, and tamper-evident incident records.

1.3 Problem Statement

Security teams face several recurring challenges:

- **Fragmented signals:** alerts and logs are distributed across tools, making context difficult to assemble.
- **Investigation overhead:** analysts must determine what happened, why it matters, and what to do next.
- **Repeated defensive work:** lessons from one environment are not always converted into reusable rules.
- **Unsafe automation:** AI-generated rules may be incorrect, overly broad, or unsafe if applied without validation.
- **Evidence gaps:** incident reviews require a traceable timeline of the event, interpretation, approvals, and actions.

1.4 Project Goals

1. Detect suspicious interaction with deliberately placed decoys.
2. Convert events into normalized, explainable threat context.
3. Recommend defensive rules and share approved rules with opted-in environments.
4. Maintain verifiable evidence of event receipt, analysis, approval, and response.
5. Demonstrate how organized evidence could support wider incident and risk-review workflows.

2 Proposed Solution

Mrigmareecha connects detection, investigation, defensive learning, and evidence management through a common workflow.

2.1 Controlled Deception

The platform can integrate with a controlled OpenCanary deployment and optionally use Canarytokens or equivalent authorized canary artifacts. These assets are placed only in environments where the team has permission to monitor activity.

When a decoy service receives an unexpected connection or a canary artifact is activated, an event is forwarded to Mrigmareecha.

The system should collect only fields needed for investigation, such as event type, timestamp, service or token identifier, permitted network metadata, and a reference to the original record.

2.2 AI-Assisted Investigation

A threat-analysis service normalizes the event and combines it with available context. It produces a structured assessment containing:

- A short explanation of what was observed.
- A severity estimate and confidence value.
- Evidence supporting the assessment.
- Plausible benign explanations or uncertainty.
- Recommended defensive steps.
- A proposed rule, if the evidence supports one.

The system distinguishes observed facts from model-generated interpretations. The model's assessment is a recommendation, not proof of attacker identity, intent, or attribution.

2.3 Collective Defensive Learning

After validation, a finding may be converted into a portable, sanitized defensive rule or indicator. Participating organizations can opt in to receive the rule, inspect its rationale, test it in a non-enforcing mode, and approve it according to their policies.

Raw logs, credentials, customer records, and unnecessary identifying information should not be shared by default.

2.4 Evidence and Risk Review

Mrigmareecha records the event, analysis version, analyst decision, rule lifecycle, and response actions in an audit trail. An authorized analyst can export a structured incident evidence bundle for internal investigation or later risk review.

The platform does not claim that a canary alert alone proves a covered cyber incident or guarantees an insurance payment.

2.5 Core Differentiators

Capability	Proposed value
Deception-first signals	Provides high-context events from controlled assets, complementing generic security alerts.
Structured AI triage	Turns event records into a consistent explanation, severity estimate, evidence list, and suggested next steps.
Reviewable rule exchange	Allows defensive patterns to travel between opted-in participants without requiring raw-log sharing.
Human-governed enforcement	Prevents an unverified model output from directly changing production security controls.
Evidence lifecycle	Preserves a traceable sequence of observations, decisions, and actions for later investigation.

3 Target Users and Use Cases

3.1 Primary Users

- **Security analysts:** need concise triage and a reliable record of investigation decisions.
- **IT and infrastructure teams:** need actionable recommendations that fit their existing environments.
- **Small and mid-sized organizations:** may benefit from structured investigation workflows without building a large custom threat-intelligence operation.
- **Cyber-risk stakeholders:** may use organized incident records as one input into a wider risk-review process.

3.2 Illustrative Scenarios

3.2.1 Unexpected Access to a Decoy Service

An authorized lab deployment of OpenCanary records a connection to a monitored decoy service. Mrigmareecha ingests the event, correlates available metadata, and presents a timeline and explanation. The analyst can mark the finding as suspicious, benign, or requiring further investigation.

3.2.2 Canary Artifact Activation

A test canary URL or artifact is accessed in a controlled exercise. The system treats the activation as a signal that the artifact was encountered. It does not assume that the event identifies a specific person or proves malicious intent.

3.2.3 Reusable Defensive Pattern

Several validated events indicate recurring behavior. The system suggests a sanitized rule with a description, supporting evidence, scope, limitations, and a test case. The rule remains a candidate until it passes validation and receives the required approval.

3.2.4 Evidence Export

An analyst exports the event timeline, original event references, analysis summary, approvals, rule versions, and integrity metadata. The bundle can support an internal review or be considered by an authorized third party alongside other required evidence.

4 System Architecture

4.1 High-Level Architecture

The architecture is divided into five logical layers:

1. **Signal layer:** controlled decoys, OpenCanary events, optional canary-token callbacks, and approved telemetry sources.
2. **Ingestion layer:** authenticated webhook or REST endpoints that validate, normalize, and deduplicate events.
3. **Analysis layer:** deterministic enrichment, risk scoring, and a replaceable AI analysis provider.
4. **Governance layer:** analyst review, rule validation, opt-in distribution, version control, and rollback.
5. **Evidence and interface layer:** dashboard, event timeline, audit log, and evidence-bundle export.

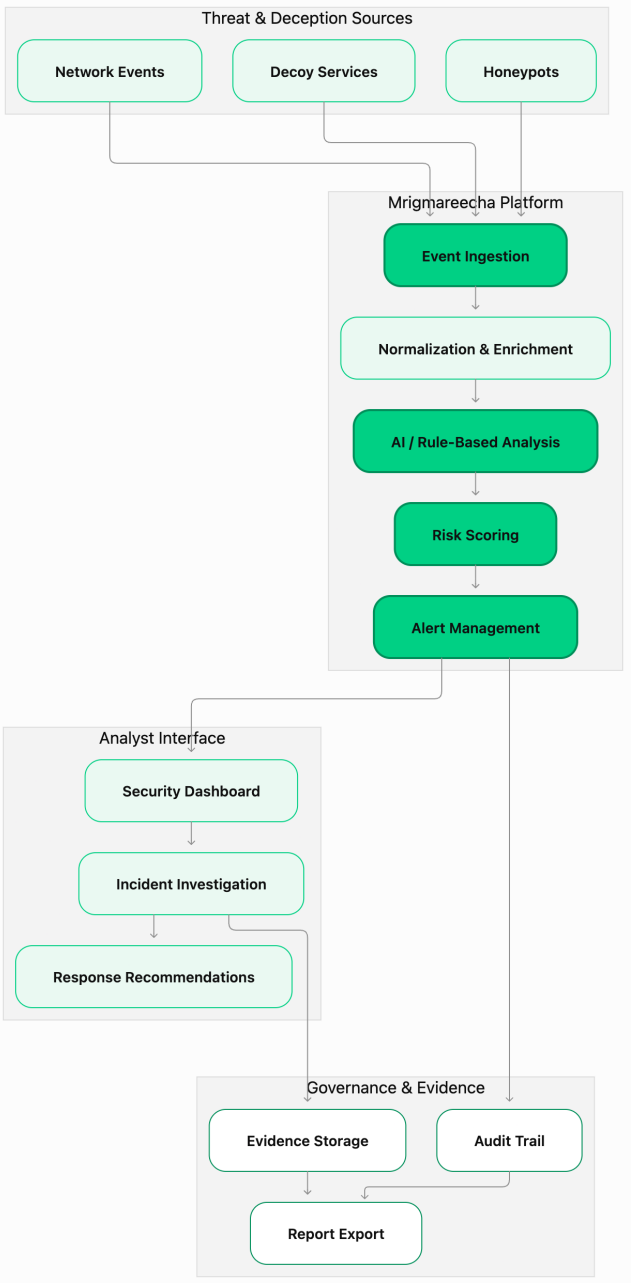


Figure 1: High-Level System Architecture

4.2 MVP Component Design

Component	MVP implementation	Responsibility
Event collector	FastAPI endpoint	Accepts authorized test events, validates the schema, and rejects malformed requests.
Normalizer	Python service	Maps incoming records to a consistent event format and applies deduplication.
Analysis provider	Rules plus optional LLM	Produces structured analysis, explanations, confidence, and defensive recommendations.
Event store	PostgreSQL	Stores events, assessments, approvals, and rule versions.
Dashboard	React or Next.js	Displays events, severity, evidence, analyst actions, and candidate rules.
Rule registry	Versioned JSON records	Tracks scope, rationale, tests, status, approvals, and rollback metadata.
Evidence exporter	Python service	Creates a JSON evidence bundle and integrity manifest.

4.3 Gordon AI Integration Boundary

Mitigata's Gordon AI is a potential integration point if an authorized API, sandbox, or integration contract is made available. This proposal does not assume access to private internals or claim that the hackathon MVP already runs on Gordon AI.

To avoid vendor lock-in, the analysis layer should expose a common interface. During the hackathon, it can use deterministic rules and an available local or API-backed language model. A Gordon AI adapter can be implemented later if access is approved.

Design principle
The event schema, evidence model, rule lifecycle, and dashboard should not depend on one LLM vendor. If the model is unavailable, the platform should still accept events, apply deterministic checks, and show a transparent fallback status.

5 End-to-End Event Lifecycle

5.1 Step-by-Step Workflow

1. **Observe:** a controlled decoy service or canary artifact generates an event in an authorized environment.
2. **Receive:** the ingestion API authenticates the source where applicable, validates the event, and assigns an internal ID.
3. **Normalize:** the system converts the source record into a consistent schema and records its origin.
4. **Enrich:** deterministic logic adds available context, such as service type, related events, and environment metadata.
5. **Analyze:** the analysis provider produces a structured assessment with evidence and uncertainty.
6. **Prioritize:** the event receives a severity estimate based on transparent rules and contextual factors.

7. **Review:** the analyst inspects the event, explanation, and recommended next steps.
8. **Propose:** when justified, the platform drafts a defensive rule or investigation action.
9. **Validate:** a candidate rule is checked against test cases and reviewed for unintended impact.
10. **Record:** the decision, rule version, and actions are added to the audit trail.

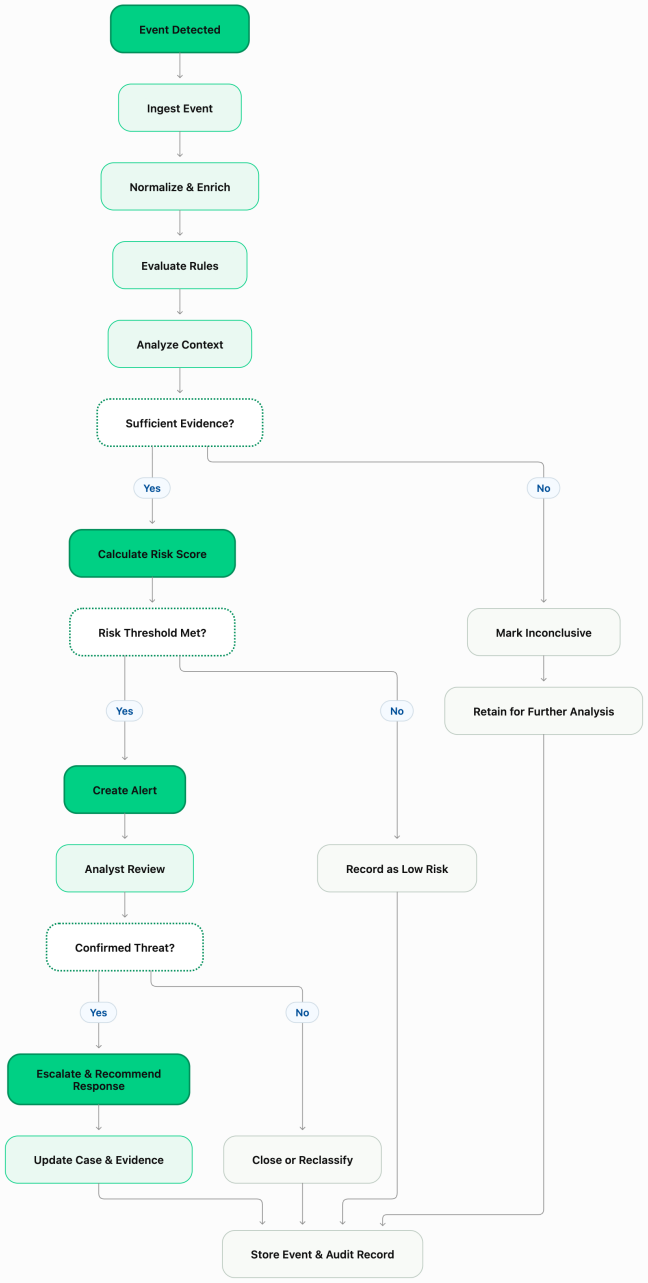


Figure 2: Event Lifecycle and Decision Workflow

5.2 Illustrative Event Record

Listing 1: Illustrative normalized event

```
{
  "event_id": "evt_demo_001",
  "source_type": "opencanary",
  "event_type": "decoy_service_connection",
  "observed_at": "2026-10-18T10:15:00Z",
  "asset_id": "lab-decoy-01",
```

```
  "environment": "authorized-demo",
  "source_metadata": {
    "remote_address": "192.0.2.25",
    "service": "ssh"
  },
  "evidence_refs": ["raw-event-demo-001"],
  "analysis_status": "pending",
  "sharing_consent": false
}
```

The example IP address belongs to a documentation-only address range and is not intended to represent a real observed actor. Real event records must follow data-minimization and retention policies.

5.3 Structured Analysis Output

Listing 2: Illustrative analysis response

```
{
  "event_id": "evt_demo_001",
  "severity": "medium",
  "confidence": 0.78,
  "summary": "Unexpected connection to a monitored decoy service.",
  "observed_facts": [
    "The decoy service recorded a connection.",
    "The event occurred in the authorized demo environment."
  ],
  "hypotheses": [
    "Possible reconnaissance or an authorized test."
  ],
  "recommended_actions": [
    "Review nearby events.",
    "Confirm whether a security test was scheduled."
  ],
  "rule_candidate": null,
  "human_review_required": true
}
```

Confidence is an internal estimate, not a calibrated probability unless calibration has been measured and documented. The model must not invent evidence or present an inferred hypothesis as an observed fact.

6 AI-Assisted Threat Analysis

6.1 Role of AI

AI assists the analyst with interpretation and consistency; it is not an unquestioned source of truth. The system combines deterministic processing with model-assisted reasoning.

6.1.1 Deterministic Processing

Before calling a model, the backend should:

- Validate required event fields and reject malformed records.
- Normalize timestamps and source identifiers.
- Remove or mask unnecessary sensitive fields.

- Deduplicate repeated events where appropriate.
- Apply transparent baseline checks and severity rules.
- Attach source references so the analyst can inspect the original evidence.

6.1.2 Model-Assisted Interpretation

The model may summarize event context, identify questions an analyst should ask, compare an event with approved threat knowledge, and suggest defensive next steps.

The model should return structured output validated against a schema. Invalid output should be rejected or routed through a safe fallback path.

6.1.3 Prompt and Data Security

Logs and external event fields must be treated as untrusted data. They may contain text intended to manipulate the model. The analysis prompt should clearly separate system instructions from event content, and application policy must be enforced in code.

- Do not let event text override system instructions.
- Do not allow the model to execute shell commands or arbitrary code against monitored systems.
- Do not pass secrets or unnecessary personal data to the model.
- Validate all model-generated rule candidates before use.
- Record the model/provider version and relevant configuration needed to reproduce the decision.
- Provide a deterministic fallback when the model is unavailable or returns invalid output.

6.2 Threat Assessment Dimensions

Dimension	Interpretation
Signal strength	How directly the event relates to a monitored deception asset.
Asset context	Whether the affected decoy or environment is sensitive within the authorized test setup.
Corroboration	Whether independent related events support the initial observation.
Recurrence	Whether similar observations have occurred within a defined time window.
Uncertainty	Whether the event may be explained by a scheduled test, scanner, misconfiguration, or another benign activity.
Actionability	Whether a safe, proportionate defensive step is supported by the available evidence.

The final severity should be determined using documented rules and reviewable factors. A model-generated score alone should not automatically trigger a disruptive action.

7 Collective Defense and Rule Sharing

7.1 From One Finding to Reusable Intelligence

A validated defensive lesson can be useful beyond the environment where it was discovered. Mrigmareecha proposes a governed mechanism for converting findings into reusable rules.

A candidate rule may contain:

- A unique rule ID and version.

- A concise description of the observed behavior.
- A rule type and intended enforcement location.
- The evidence and rationale supporting the rule.
- Confidence, scope, limitations, and known false positives.
- A test case and validation result.
- Approval state and change history.
- Creation, review, expiry, or revocation timestamps.

7.2 Proposed Rule Lifecycle

1. **Draft:** the platform creates a candidate from a reviewed finding.
2. **Sanitize:** source-specific identifiers and sensitive details are removed unless sharing them is explicitly justified and authorized.
3. **Validate:** the rule is checked for schema correctness, scope, conflicts, and expected behavior.
4. **Review:** an authorized reviewer approves, edits, or rejects the rule.
5. **Publish:** the approved version is made available to participating organizations that have opted in.
6. **Test locally:** recipients run the rule in a non-enforcing or monitoring mode before considering enforcement.
7. **Promote or roll back:** each organization controls its own deployment and can disable or revert the rule.

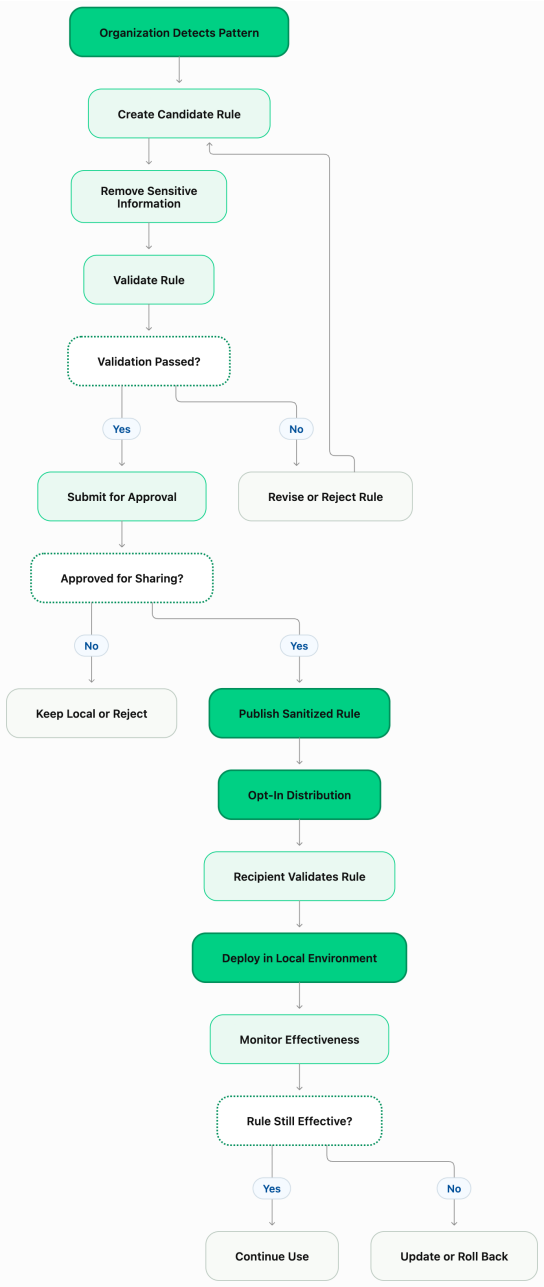


Figure 3: Collective Rule-Sharing Lifecycle

7.3 Example Rule Representation

Listing 3: Illustrative defensive rule metadata

```
{
  "rule_id": "mr-decoy-connection-review",
  "version": 1,
  "rule_type": "analyst_detection",
  "status": "candidate",
  "description": "Review unexpected access to a monitored decoy.",
  "scope": {
    "source_type": "opencanary",
    "environment": "authorized"
  },
  "rationale": "A decoy interaction merits contextual review.",
  "validation": {
    "schema_valid": true,
```

```
    "test_status": "pending"
  },
  "distribution": {
    "opt_in_required": true,
    "raw_event_sharing": false
  },
  "enforcement": {
    "automatic": false,
    "human_approval_required": true
  }
}
```

This example is a rule-registry record, not a production firewall rule. Concrete rules for tools such as WAFs, SIEMs, or network detection systems require separate syntax checks, test fixtures, environment-specific review, and rollback plans.

7.4 Why Not Automatically Push Every Rule?

Automatically deploying unverified AI-generated rules across multiple organizations could create false positives, service disruption, or a shared failure mode. Mrigmareecha separates rule generation from approval and enforcement.

The MVP will demonstrate a simulated rule registry and review workflow. It will not push live firewall, eBPF, endpoint, or production network changes.

8 Evidence Integrity and Cyber-Risk Workflows

8.1 Evidence as a First-Class Output

A security alert is more useful when an analyst can trace how it was generated, how it was interpreted, and what happened afterward. Mrigmareecha proposes an evidence bundle that links the original event to its analysis, approvals, and related rule versions.

A bundle may include:

- Event IDs, timestamps, source type, and source-record references.
- A copy or reference to permitted original evidence.
- Normalization and enrichment metadata.
- Analysis summary, provider version, and uncertainty.
- Analyst comments, decision, and approval timestamp.
- Candidate and approved rule versions with test results.
- Response-action records and rollback information, if any.
- Hashes and a manifest for integrity verification.

8.2 Tamper-Evident Audit Trail

For the MVP, the system can generate a SHA-256 digest for each canonicalized audit record and link it to the previous record's hash. An exported manifest can include the record hashes and the final chain hash.

This can help reveal changes to records after hashing. However, a hash chain does not prove that the original event was truthful, that the system clock was correct, or that the event was collected independently. Those properties require additional controls and reliable source handling.

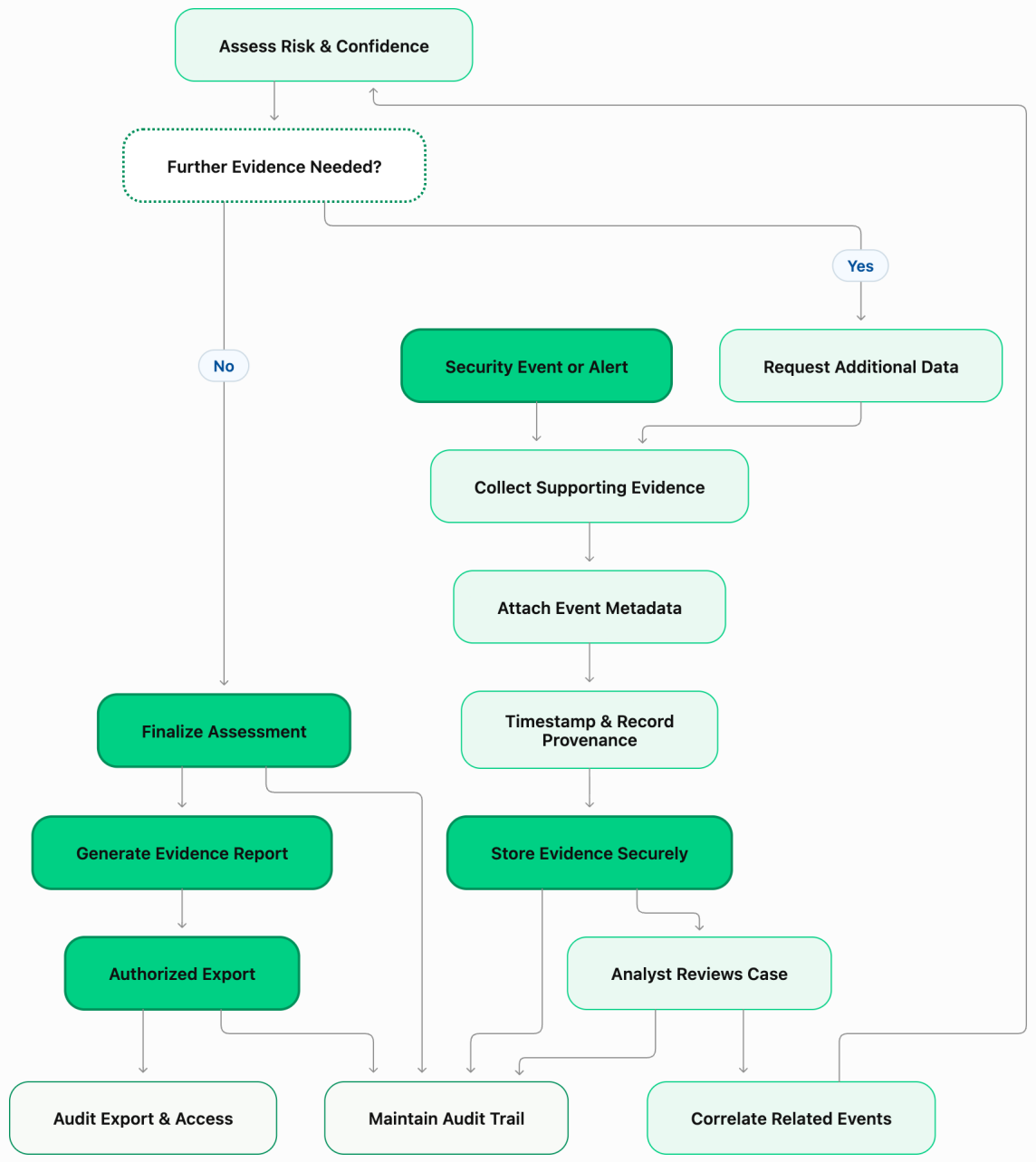


Figure 4: Evidence Collection and Risk Review

8.3 Parametric Insurance: Future Extension

Mrigmareecha could eventually provide structured telemetry to a separately governed cyber-risk or parametric-insurance workflow. A future system might evaluate whether independently verified signals satisfy a defined trigger in an approved policy.

That capability is outside the core MVP. A canary activation alone does not establish the scope of loss, policy coverage, causation, or eligibility for payment.

A real insurance integration would require:

- Explicit policy definitions and agreed trigger conditions.
- Independent validation of relevant telemetry.
- Authentication, anti-replay, and fraud controls.
- Clear evidence-retention and dispute-handling procedures.
- Approval from the insurer and other relevant stakeholders.

- Legal, regulatory, privacy, and underwriting review.

The hackathon demonstration may show a **simulated trigger evaluation** using synthetic test data. It must clearly label the result as a simulation, not a policy decision, claim approval, or promise of payment.

9 Hackathon MVP Scope

9.1 What We Will Build

The MVP is designed to demonstrate one end-to-end, repeatable defensive workflow instead of a large collection of loosely connected features.

Feature	Priority	Acceptance condition
Event ingestion	Must have	The backend accepts a valid test event and rejects malformed input.
Event normalization	Must have	Incoming records appear in a consistent schema with a unique ID.
Threat analysis	Must have	The platform produces a summary, severity, evidence, uncertainty, and recommended next steps.
Analyst dashboard	Must have	A user can inspect events and view their analysis and status.
Rule candidate	Must have	The system creates a candidate rule from a reviewed demo finding.
Approval workflow	Must have	A reviewer can approve, reject, or defer a candidate.
Audit trail	Must have	The system records event and analyst actions in a traceable sequence.
Evidence export	Should have	An analyst can export a JSON bundle and hash manifest.
OpenCanary integration	Should have	A controlled lab event reaches the backend through an adapter.
Collective-sharing simulation	Should have	A sanitized rule appears in a simulated opt-in recipient view.
Gordon AI integration	Conditional	Implement only if authorized API or sandbox access is provided.
Live production enforcement	Out of scope	No automatic production firewall, endpoint, or network changes.

9.2 Proposed Technology Stack

Layer	Technology and rationale
Backend API	Python and FastAPI for REST endpoints, validation, and service orchestration.
Deception source	OpenCanary in an isolated, authorized lab; optional Canarytokens integration where appropriate.
Analysis	Deterministic Python rules plus an interchangeable local or API-backed LLM provider.
Data store	PostgreSQL for events, assessments, approvals, and versioned rules.
Frontend	React or Next.js for the analyst dashboard.
Packaging	Docker Compose for repeatable local development and demonstration.
Integrity	SHA-256 hashes over canonicalized audit records and an export manifest.
Future scaling	A queue, columnar analytics store, or event streaming system only when workload and deployment needs justify it.

10 Implementation Plan

10.1 Suggested Build Sequence

Phase	Deliverable
Phase 1: Define	Agree on event schema, rule schema, demo scenario, and API contracts.
Phase 2: Core Pipeline	Implement ingestion, normalization, storage, deterministic fallback, and a sample analysis response.
Phase 3: Dashboard	Connect the frontend to the event and assessment APIs.
Phase 4: Governance	Add candidate-rule creation, review states, and audit records.
Phase 5: Evidence	Generate an exportable bundle and verify the hash manifest.
Phase 6: Integration	Connect the lab source if feasible; test synthetic events as fallback.
Phase 7: Stabilize	Run the demo from a clean environment, verify failure paths, and prepare the pitch.

Scope-control rule
If time becomes limited, preserve the end-to-end workflow: ingest one event, explain it, review it, create one candidate rule, record the decision, and export evidence. Defer multi-tenant infrastructure, production enforcement, and insurer integrations rather than leaving the core path incomplete.

11 Security, Privacy, and Governance

A security product must be safe by design. Mrigmareecha should operate only on authorized assets and should never perform retaliatory actions against a suspected source.

11.1 Threat Model

Risk	Mitigation
Forged events	Authenticate ingestion sources where feasible, validate schemas, and preserve source provenance.
Prompt injection in logs	Treat event content as untrusted data, isolate it from system instructions, and validate model output.
Incorrect AI assessment	Expose supporting facts and uncertainty, use deterministic fallback, and require human review for consequential actions.
Unsafe defensive rule	Use schema checks, test fixtures, limited scope, human approval, monitoring mode, versioning, and rollback.
Cross-tenant leakage	Apply opt-in sharing, data minimization, tenant authorization, sanitization, and recipient-side access controls.
Evidence tampering	Use canonical serialization, hashes, restricted write access, and a traceable audit history.
False attribution	Avoid identity claims unsupported by evidence; treat network metadata and canary activation as contextual signals only.
Service disruption	Do not enable automatic production blocking or enforcement in the hackathon MVP.
Data over-retention	Define retention limits and collect only fields needed for the authorized security purpose.

11.2 Operational Guardrails

- Use synthetic credentials and decoy data, never real secrets.
- Deploy decoys only in an isolated or explicitly authorized environment.
- Do not execute attacker-supplied commands or payloads.
- Do not scan, probe, or retaliate against third-party systems.
- Keep model outputs separate from source evidence.
- Require approval before sharing a rule or taking a consequential defensive action.
- Record who approved each rule and which version was used.
- Provide a way to revoke shared rules and disable integrations.

11.3 Privacy-Aware Sharing

Collective defense must not become indiscriminate data collection. The default shared object should be a sanitized behavioral pattern or defensive rule, not the original event record.

Before publication, the system should remove or generalize direct identifiers that are not necessary for the rule's intended purpose. Any sharing of raw records, specific indicators, or sensitive metadata requires an explicit policy and appropriate authorization.

12 Expected Impact and Evaluation

Mrigmareecha aims to improve the consistency and speed of the investigation workflow while making defensive learning more reusable. These are intended outcomes to test, not performance claims that have already been proven.

12.1 Evaluation Metrics

Metric	How it will be evaluated
Ingestion reliability	Percentage of valid test events successfully stored and displayed.
Processing latency	Time from event submission to the first available assessment.
Assessment completeness	Percentage of assessments containing the required structured fields.
Evidence traceability	Whether an analyst can navigate from a finding to the source reference, analysis, decision, and rule version.
Rule-validation quality	Percentage of candidate rules passing defined schema and test-case checks before approval.
Human-review compliance	Whether the system blocks rule promotion when required approval is missing.
Export integrity	Whether changes to a hashed audit record can be detected during manifest verification.
Sharing privacy	Whether the shared rule omits fields excluded by the configured sharing policy.

12.2 Proposed Success Criteria

1. A valid event can be ingested and viewed end to end.
2. The assessment includes facts, hypotheses, uncertainty, and actionable recommendations.
3. The analyst can approve or reject a candidate rule.
4. An unapproved candidate cannot be promoted through the normal workflow.
5. The recipient view receives only the sanitized, approved version in the sharing simulation.
6. An evidence bundle can be exported and its integrity manifest verified.
7. A malformed event or unavailable model produces a safe, understandable failure state.

12.3 Potential Real-World Value

If validated in pilot deployments, the workflow could help teams reduce repetitive triage work, preserve a more consistent audit trail, and reuse defensive knowledge across environments. Actual time savings, detection quality, cost reduction, and adoption would need to be measured with representative users and workloads.

13 Alignment with CyberForge

The project is designed around the challenge theme of applying AI to practical cybersecurity problems.

Challenge dimension	Mrigmareecha response
AI-assisted security operations	Uses AI to summarize and prioritize authorized security events and recommend next steps.
Threat detection	Collects signals from controlled deception assets and enriches them with available context.
Actionable automation	Creates candidate defensive rules with validation and human approval instead of uncontrolled automatic deployment.
Security collaboration	Supports opt-in distribution of sanitized, versioned defensive knowledge.
Incident investigation	Links event references, assessments, decisions, and response history in an exportable evidence bundle.
Responsible AI	Treats model output as fallible, validates structured responses, and preserves human control over consequential actions.

The concept is modular. If an approved integration with Mitigata's platform or Gordon AI becomes available, it can be connected through the analysis or event-ingestion adapter without requiring the entire application to be redesigned.

14 Conclusion

Mrigmareecha proposes a practical way to connect four stages of defensive security: controlled deception, AI-assisted analysis, governed intelligence sharing, and evidence preservation.

Its central contribution is not simply using an LLM to summarize an alert. It is the workflow around that analysis: preserve source evidence, separate facts from hypotheses, keep defensive rules reviewable, require approval before consequential actions, and record how a decision was reached.

For CyberForge, Team 360degree will focus on demonstrating one complete and testable path from a controlled signal to a reviewed defensive recommendation and an exportable evidence record. Advanced integrations, including Gordon AI and insurance-related workflows, remain explicit extensions requiring appropriate access, validation, and governance.

MRIGMAREECHA

Detect with deception. Understand with AI.
Defend together.

Team 360degree
Vipul Mhatre | Samarth Pachpile

`github.com/Vipul-Mhatre/Mrigmareecha`

References and Implementation Notes

The following resources provide starting points for implementation and security design. Their inclusion does not imply a formal partnership or endorsement.

1. **OpenCanary** — open-source network threat-detection honeypot.
   https://github.com/thinkst/opencanary
2. **Canarytokens** — canary artifacts and alerting resources. Select a deployment model appropriate to the project and review its service and privacy terms.
   https://canarytokens.org/

3. **MITRE D3FEND** — knowledge base of defensive cybersecurity techniques.
https://d3fend.mitre.org/
4. **NIST SP 800-53 Rev. 5** — security and privacy control catalog useful as a broader reference for control design.
https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final
5. **Mitigata** — company and product context.
https://mitigata.com/
6. **Mrigmareecha GitHub Repository** — project source code and implementation.
`github.com/Vipul-Mhatre/Mrigmareecha`

Assumptions and Scope Notes

- OpenCanary and all monitored assets will be configured only in a lab or environment where the team has explicit authorization.
- Canarytoken functionality depends on the selected service or deployment. An activation is a contextual signal, not proof of identity or malicious intent.
- Gordon AI access, API availability, and integration permissions are not assumed. The adapter is a proposed extension.
- Collective-sharing and recipient behavior in the MVP may be simulated with synthetic organizations and data.
- Any parametric-insurance workflow is a future concept; the MVP will not issue policies, approve claims, or make payments.
- No unverified performance, market-size, premium-discount, or deployment-scale figures are claimed in this proposal.